

Titanic Data Lab

NumPy + Pandas from Zero to Feature Engineering and Hypothesis Testing

AI Foundation Program

Instructor Learning Guide

Contents

Course Positioning	6
1. What Are We Going to Learn?	6
2. Download the Dataset	7
3. Import the Libraries	7
Part I - NumPy Foundations	7
4. What is NumPy?	7
5. Python List vs NumPy Array	7
6. Why NumPy?	8
7. NumPy Array Properties	8
8. NumPy Indexing	8
9. NumPy Slicing	9
10. Mathematical Operations	9
11. NumPy Aggregation	9
12. Boolean Operations	9
13. Boolean Filtering	9
14. Multiple Conditions	10
15. NumPy where	10
16. Missing Data in NumPy	10
Part II - Pandas Foundations with Titanic	10
17. Load the Real Titanic Dataset	10
18. What is Pandas?	10
19. What is a Series?	11
20. What is a DataFrame?	11

21. First Look at the Dataset	11
22. Look at the Last Rows	11
23. Random Sample	11
24. Dataset Shape	11
25. Column Names	11
26. What Do These Columns Mean?	12
27. <code>info()</code> - One of the Most Important Commands	12
28. Data Types	12
29. Numerical Summary	12
30. Include Categorical Variables	13
31. Selecting One Column	13
32. Selecting Multiple Columns	13
33. Row Selection with <code>iloc</code>	13
34. Row Selection with <code>loc</code>	13
35. Filtering Data	13
36. Filter by Sex	14
37. Multiple Conditions	14
38. OR Conditions	14
39. <code>isin()</code>	14
40. Sorting Data	14
41. Count Unique Values	14
42. Convert Counts to Percentages	14
43. Survival Rate	15
44. <code>groupby()</code> - Extremely Important	15
45. Multiple <code>groupby()</code> Variables	15
46. Multiple Aggregations	15
47. Group Fare Statistics	15
48. Cross Tabulation	15
49. Survival by Passenger Class	16
50. Missing Values	16
51. Missing-Value Percentage	16

52. Visualize Missingness	16
53. Why is Missing Data Important?	16
54. Strategy 1 - Drop Missing Values	17
55. Drop Rows Missing Age Only	17
56. Strategy 2 - Fill Missing Values	17
57. Why Median Instead of Mean?	17
58. Better Age Imputation	17
59. Missingness as a Feature	18
60. Fill embarked	18
61. What Should We Do with deck?	18
62. Duplicates	18
Part III - Exploratory Data Analysis	19
63. What is Exploratory Data Analysis?	19
64. Question 1 - How Many Survived?	19
65. Plot Survival	19
66. Question 2 - Survival and Sex	19
67. Question 3 - Passenger Class and Survival	19
68. Question 4 - Age Distribution	20
69. Survivors vs Non-Survivors - Age	20
70. Question 5 - Fare	20
Part IV - Feature Engineering	20
71. What is Feature Engineering?	20
72. Feature Engineering 1 - Family Size	21
73. Family Size and Survival	21
74. Feature Engineering 2 - Travelling Alone	21
75. Compare with Existing aLone	21
76. Feature Engineering 3 - Age Groups	22
77. Age Group Distribution	22
78. Feature Engineering 4 - Child	22
79. Feature Engineering 5 - Family Type	22
80. Family Type and Survival	23

81. Feature Engineering 6 - Fare Per Person	23
82. Feature Engineering 7 - Fare Bands	23
83. cut () vs qcut ()	24
84. Feature Engineering 8 - Fare Log Transformation	24
85. Compare Fare Distribution	24
86. Categorical Data	24
87. Binary Encoding	25
88. One-Hot Encoding	25
89. Understanding Redundant Features	25
90. The Most Important Example - Data Leakage	26
91. Leakage vs Redundancy	26
92. Build a Clean Working Dataset	26
93. Add Missing-Value Indicator	27
94. Impute Age	27
95. Fill Embarkation	27
96. Add Family Size	27
97. Add Alone Feature	27
98. Add Child Feature	27
99. Add Fare Per Person	27
100. Add Age Groups	28
101. Inspect the Engineered Dataset	28
Part V - Hypothesis-Driven Data Analysis	28
102. What is a Hypothesis?	28
103. Scientific Thinking	28
104. Null Hypothesis	29
105. Alternative Hypothesis	29
106. What is a p-value?	29
107. Statistical Significance is Not Practical Importance	29
108. Hypothesis 1 - Sex and Survival	30
109. Create the Contingency Table	30
110. Run the Chi-Square Test	30

111. Interpret the Test	30
112. Effect Size - Cramer's V	30
113. Hypothesis 2 - Passenger Class and Survival	31
114. Do Not Stop at the p-value	31
115. Hypothesis 3 - Age and Survival	32
116. Compare Descriptive Statistics First	32
117. Welch's t-test	32
118. Important Assumptions	33
119. Hypothesis 4 - Fare and Survival	33
120. Mann-Whitney U Test	34
121. Hypothesis 5 - Travelling Alone and Survival	34
122. Feature Engineering Can Create New Hypotheses	34
123. Test Family Type and Survival	35
124. Association Does Not Mean Causation	35
125. Example of Confounding	36
126. Ask Better Questions	36
127. Correlation	36
128. Correlation with Survival	37
Part VI - From Analysis to Machine-Learning Data	37
129. A Complete Data Analysis Workflow	37
130. Prepare a Machine-Learning Dataset	37
131. Encode Categorical Variables	38
132. Check Everything is Numerical	38
133. Separate Features and Target	38
134. What Did We Actually Do?	38
135. Important NumPy Commands Learned	39
136. Important Pandas Commands Learned	39
137. Important Statistics Learned	40
138. The Most Important Thinking Skill	40
Part VII - Guided Student Exercises	41
139. Guided Student Exercise	41

140. Advanced Student Challenge 42

141. Feature Engineering Challenge 42

142. Hypothesis Challenge 43

 Hypothesis A 43

 Hypothesis B 43

 Hypothesis C 43

143. Final Mini Project - Titanic Data Investigation 43

 Section 1 - Dataset 43

 Section 2 - NumPy 44

 Section 3 - Pandas 44

 Section 4 - Data Cleaning 44

 Section 5 - EDA 44

 Section 6 - Feature Engineering 44

 Section 7 - Hypothesis Testing 45

 Section 8 - Conclusions 45

144. Final Lesson 45

145. Main Takeaway 46

Instructor Emphasis 46

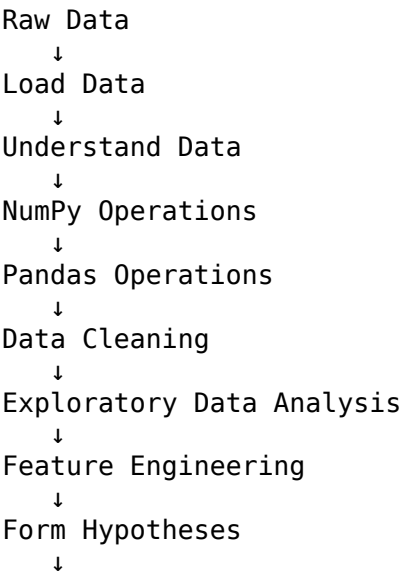
Course Positioning

This lab is designed as a practical bridge from Python foundations into data understanding for AI. It uses a single real dataset throughout so learners can connect syntax, statistics, data preparation, feature engineering, and evidence-based reasoning.

- Dataset:** Titanic passenger dataset from the Seaborn repository
- Main goal:** Learn how raw data becomes useful information and eventually useful features for machine learning.

1. What Are We Going to Learn?

In this lesson we will use one real dataset to understand the complete beginner data-analysis workflow:



Test Hypotheses

↓

Prepare Data for Machine Learning

By the end, students should understand:

- What NumPy is and why arrays matter
- Vectorized operations, indexing, slicing, filtering, and aggregation
- Missing numerical values
- What pandas Series and DataFrames are
- Loading and inspecting CSV data
- Selecting, filtering, sorting, grouping, and aggregating
- Missing-value handling and exploratory data analysis
- Feature engineering and categorical encoding
- Redundant features and data leakage
- Null and alternative hypotheses
- Chi-square tests, Welch's t-tests, and Mann-Whitney U tests
- Effect size, confounding, and association versus causation
- How to prepare a model-ready feature table

2. Download the Dataset

If you are using Google Colab:

```
!wget https://raw.githubusercontent.com/mwaskom/seaborn-data/master/titanic.csv
```

Check whether the file exists:

```
!ls
```

You should see:

```
titanic.csv
```

3. Import the Libraries

We mainly need:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

For statistical hypothesis testing:

```
from scipy import stats
```

Part I - NumPy Foundations

4. What is NumPy?

NumPy stands for **Numerical Python**. It helps Python work efficiently with numerical data.

Instead of processing values one by one, NumPy allows us to perform operations on many values together.

5. Python List vs NumPy Array

Start with a normal Python list:

```
ages = [22, 38, 26, 35, 35]
```

```
print(ages)
print(type(ages))
```

Create a NumPy array:

```
ages_array = np.array(ages)
```

```
print(ages_array)
print(type(ages_array))
```

The values may look similar, but `ages_array` is a `numpy.ndarray`, not a Python `list`.

6. Why NumPy?

Imagine increasing every age by 1.

With a normal Python list:

```
ages = [22, 38, 26, 35, 35]
new_ages = []
```

```
for age in ages:
    new_ages.append(age + 1)
```

```
print(new_ages)
```

With NumPy:

```
ages = np.array([22, 38, 26, 35, 35])
new_ages = ages + 1
print(new_ages)
```

This is **vectorization**: applying an operation to an entire array without manually writing a Python loop.

7. NumPy Array Properties

```
numbers = np.array([10, 20, 30, 40, 50])
```

Check its properties:

```
numbers.shape
numbers.ndim
numbers.dtype
numbers.size
```

- `shape` tells us the structure of the array.
- `ndim` tells us how many dimensions it has.
- `dtype` tells us the stored data type.
- `size` tells us how many values are stored.

8. NumPy Indexing

```
numbers = np.array([10, 20, 30, 40, 50])
```

```
numbers[0]    # first element
numbers[1]    # second element
numbers[-1]   # last element
```

9. NumPy Slicing

```
numbers = np.array([10, 20, 30, 40, 50])
```

```
numbers[0:3]  
numbers[2:]
```

Slicing lets us select a range of values from an array.

10. Mathematical Operations

```
numbers = np.array([10, 20, 30, 40, 50])
```

```
numbers + 10  
numbers * 2  
numbers / 10  
numbers ** 2
```

Each operation is applied element-by-element.

11. NumPy Aggregation

Aggregation reduces many values into summary information.

```
numbers = np.array([10, 20, 30, 40, 50])
```

```
np.mean(numbers)  
np.median(numbers)  
np.min(numbers)  
np.max(numbers)  
np.sum(numbers)  
np.std(numbers)
```

These are core operations for data analysis and statistics.

12. Boolean Operations

```
ages = np.array([22, 38, 16, 35, 12, 40])  
ages >= 18
```

The result is a Boolean mask similar to:

```
[ True  True False  True False  True]
```

13. Boolean Filtering

Use a Boolean condition to retrieve the actual values:

```
ages[ages >= 18]  
ages[ages < 18]
```

This pattern becomes extremely important in pandas.

14. Multiple Conditions

```
ages = np.array([5, 15, 22, 38, 65, 80])
```

```
ages[(ages >= 18) & (ages <= 60)]
```

With NumPy arrays, use & for AND and | for OR when combining element-wise conditions.

15. NumPy where

Sometimes we want to create a category from a condition:

```
ages = np.array([5, 15, 22, 38, 65, 80])
```

```
category = np.where(ages >= 18, "Adult", "Child")  
print(category)
```

This is our first simple example of **feature engineering**: converting raw values into a new representation.

16. Missing Data in NumPy

NumPy commonly represents missing numerical values using `np.nan`.

```
ages = np.array([22, 38, np.nan, 35, 28])
```

A normal mean may return nan:

```
np.mean(ages)
```

Use missing-aware operations when appropriate:

```
np.nanmean(ages)  
np.nanmedian(ages)  
np.nanmin(ages)  
np.nanmax(ages)
```

Part II - Pandas Foundations with Titanic

17. Load the Real Titanic Dataset

```
import pandas as pd
```

```
df = pd.read_csv("titanic.csv")
```

18. What is Pandas?

Pandas is a Python library designed for working with structured data.

The two most important pandas structures are:

- **Series** - usually one column
- **DataFrame** - a table containing rows and columns

19. What is a Series?

```
df["age"]  
type(df["age"])
```

A single column is normally represented as a pandas Series.

20. What is a DataFrame?

```
type(df)
```

The Titanic table is stored in a pandas DataFrame.

21. First Look at the Dataset

Never start analysing data before looking at it.

```
df.head()  
df.head(10)
```

22. Look at the Last Rows

```
df.tail()
```

23. Random Sample

head() only shows the first few records. A random sample can give a broader impression.

```
df.sample(5)
```

24. Dataset Shape

```
df.shape
```

The result is:

```
(rows, columns)
```

The Seaborn Titanic dataset contains 891 passenger rows and 15 columns.

25. Column Names

```
df.columns
```

Columns include:

```
survived  
pclass  
sex  
age  
sibsp  
parch  
fare  
embarked  
class
```

who
adult_male
deck
embark_town
alive
alone

26. What Do These Columns Mean?

Column	Meaning
survived	Whether passenger survived
pclass	Passenger class
sex	Sex
age	Age
sibsp	Number of siblings/spouses aboard
parch	Number of parents/children aboard
fare	Ticket fare
embarked	Port of embarkation code
class	Passenger class as text
who	Man / woman / child
adult_male	Whether adult male
deck	Cabin deck
embark_town	Full embarkation town
alive	Yes/no representation of survival
alone	Whether passenger travelled alone

27. info() - One of the Most Important Commands

`df.info()`

This tells us:

- column names
- number of rows
- number of non-null values
- likely missing values
- data types
- approximate memory usage

28. Data Types

`df.dtypes`

Common pandas types include:

int64
float64
object
bool
category

29. Numerical Summary

`df.describe()`

This provides summary values such as:

```
count
mean
std
min
25%
50%
75%
max
```

30. Include Categorical Variables

```
df.describe(include="all")
```

This broadens the summary to include categorical columns.

31. Selecting One Column

```
df["age"]
```

32. Selecting Multiple Columns

```
df[["age", "sex", "fare"]]
```

Important difference:

```
df["age"]          # Series
```

```
df[["age"]]        # DataFrame
```

33. Row Selection with `iloc`

`iloc` works with numerical positions.

```
df.iloc[0]
df.iloc[0:5]
df.iloc[0:5, 0:4]
```

34. Row Selection with `loc`

`loc` works with labels and conditions.

```
df.loc[0, "age"]
df.loc[0:5, ["age", "sex", "survived"]]
```

35. Filtering Data

Passengers who survived:

```
df[df["survived"] == 1]
```

Passengers who did not survive:

```
df[df["survived"] == 0]
```

36. Filter by Sex

```
df[df["sex"] == "female"]
```

37. Multiple Conditions

Female passengers in first class:

```
df[(df["sex"] == "female") &
   (df["pclass"] == 1)]
```

Passengers older than 50 who survived:

```
df[(df["age"] > 50) &
   (df["survived"] == 1)]
```

38. OR Conditions

Passengers who were first class or second class:

```
df[(df["pclass"] == 1) |
   (df["pclass"] == 2)]
```

39. isin()

A cleaner way to select multiple categories:

```
df[df["pclass"].isin([1, 2])]
```

40. Sorting Data

Oldest passengers:

```
df.sort_values("age", ascending=False).head(10)
```

Cheapest fares:

```
df.sort_values("fare").head(10)
```

41. Count Unique Values

```
df["sex"].value_counts()
df["pclass"].value_counts()
df["survived"].value_counts()
```

42. Convert Counts to Percentages

```
df["survived"].value_counts(normalize=True)
df["survived"].value_counts(normalize=True) * 100
```

43. Survival Rate

Because survived uses 0 and 1, the mean equals the proportion of 1s.

```
df["survived"].mean()  
df["survived"].mean() * 100
```

For example, the mean of 0, 1, 1, 0, 1 is $3 / 5 = 0.60$, meaning 60% of those observations are 1.

44. groupby() - Extremely Important

Did survival differ between males and females?

```
df.groupby("sex")["survived"].mean()
```

By passenger class:

```
df.groupby("pclass")["survived"].mean()
```

45. Multiple groupby() Variables

```
df.groupby(["sex", "pclass"])["survived"].mean()
```

This asks a more detailed question: what was survival like for each combination of sex and passenger class?

46. Multiple Aggregations

```
df.groupby("pclass")["age"].agg([  
    "count",  
    "mean",  
    "median",  
    "min",  
    "max"  
])
```

47. Group Fare Statistics

```
df.groupby("pclass")["fare"].agg([  
    "count",  
    "mean",  
    "median",  
    "min",  
    "max"  
])
```

Ask students: **Why might the mean fare be different from the median fare?** This introduces skewness and outliers.

48. Cross Tabulation

```
pd.crosstab(df["sex"], df["survived"])
```

Convert counts into within-group percentages:

```
pd.crosstab(
    df["sex"],
    df["survived"],
    normalize="index"
) * 100
```

49. Survival by Passenger Class

```
pd.crosstab(
    df["pclass"],
    df["survived"],
    normalize="index"
) * 100
```

50. Missing Values

Real-world data is rarely perfectly complete.

```
df.isnull().sum()
```

Alternative:

```
df.isna().sum()
```

51. Missing-Value Percentage

```
missing_percentage = (
    df.isna().sum() / len(df) * 100
)
```

```
missing_percentage.sort_values(ascending=False)
```

Percentages are often easier to interpret than raw counts because they show the scale of the problem.

52. Visualize Missingness

```
missing_percentage.sort_values().plot(
    kind="barh",
    figsize=(8, 6)
)

plt.xlabel("Missing Values (%)")
plt.title("Missing Data in Titanic Dataset")
plt.show()
```

53. Why is Missing Data Important?

Examples in this dataset include missing age, deck, and embarkation information.

Do not assume:

```
missing = useless
```

Instead ask:

Why might this value be missing, and could the fact that it is missing itself contain information?

54. Strategy 1 - Drop Missing Values

```
df.dropna()
```

But be careful. This may remove a large portion of the dataset.

Compare:

```
len(df)
len(df.dropna())
```

Ask: **How much data did we lose?**

55. Drop Rows Missing Age Only

```
df.dropna(subset=["age"])
```

Dropping can be reasonable in some situations, but every removal should have a reason.

56. Strategy 2 - Fill Missing Values

Median age:

```
df["age"].median()
```

Create a working copy:

```
data = df.copy()
```

Fill missing age:

```
data["age"] = data["age"].fillna(
    data["age"].median()
)
```

Check:

```
data["age"].isna().sum()
```

57. Why Median Instead of Mean?

Compare:

```
df["age"].mean()
df["age"].median()
```

The median is generally more resistant to extreme values. However, median imputation is not automatically correct for every dataset. The choice should depend on the variable, distribution, missingness pattern, and analysis goal.

58. Better Age Imputation

Instead of giving every passenger the same median age, calculate a group-level median.

```
data = df.copy()
```

```
group_age_median = data.groupby(
    ["pclass", "sex"]
)["age"].transform("median")
```

```
data["age"] = data["age"].fillna(  
    group_age_median  
)
```

Now the imputed age depends on passenger class and sex.

59. Missingness as a Feature

Before filling age, preserve information about whether the value was originally missing:

```
data = df.copy()
```

```
data["age_missing"] = data["age"].isna().astype(int)
```

Inspect:

```
data[["age", "age_missing"]].head(10)
```

A model can now potentially learn whether age missingness itself carries information.

60. Fill embarked

Check category frequencies:

```
data["embarked"].value_counts()
```

Get the mode:

```
data["embarked"].mode()
```

Fill missing values:

```
data["embarked"] = data["embarked"].fillna(  
    data["embarked"].mode()[0]  
)
```

61. What Should We Do with deck?

Check missing percentage:

```
data["deck"].isna().mean() * 100
```

Possible options:

1. Drop the variable.
2. Create an Unknown category.
3. Investigate whether missingness itself is meaningful.

For example:

```
data["deck_known"] = data["deck"].notna().astype(int)
```

62. Duplicates

Check:

```
data.duplicated().sum()
```

Be careful with interpretation. Two passenger rows can theoretically have identical values across the limited columns we have. `duplicated()` does not automatically prove that a record is erroneous.

Part III - Exploratory Data Analysis

63. What is Exploratory Data Analysis?

EDA - Exploratory Data Analysis means exploring the data before modelling. The goal is to understand distributions, patterns, missingness, relationships, surprising values, and possible data-quality problems.

64. Question 1 - How Many Survived?

```
data["survived"].value_counts()
```

Percentages:

```
data["survived"].value_counts(normalize=True) * 100
```

65. Plot Survival

```
data["survived"].value_counts().plot(
    kind="bar"
)
```

```
plt.title("Titanic Survival")
plt.xlabel("Survived")
plt.ylabel("Passengers")
plt.show()
```

66. Question 2 - Survival and Sex

```
data.groupby("sex")["survived"].mean()
```

Visualize:

```
data.groupby("sex")["survived"].mean().plot(
    kind="bar"
)
```

```
plt.ylabel("Survival Rate")
plt.title("Survival Rate by Sex")
plt.show()
```

67. Question 3 - Passenger Class and Survival

```
data.groupby("pclass")["survived"].mean()
```

Plot:

```
data.groupby("pclass")["survived"].mean().plot(
    kind="bar"
)
```

```
plt.ylabel("Survival Rate")
plt.title("Survival Rate by Passenger Class")
plt.show()
```

68. Question 4 - Age Distribution

```
data["age"].hist(
    bins=30,
    figsize=(8, 5)
)

plt.xlabel("Age")
plt.ylabel("Passengers")
plt.title("Passenger Age Distribution")
plt.show()
```

69. Survivors vs Non-Survivors - Age

```
data[data["survived"] == 1]["age"].describe()
```

Compare:

```
data[data["survived"] == 0]["age"].describe()
```

Or summarize both groups:

```
data.groupby("survived")["age"].agg([
    "mean",
    "median",
    "std",
    "min",
    "max"
])
```

70. Question 5 - Fare

```
data["fare"].describe()
```

Visualize:

```
data["fare"].hist(
    bins=40,
    figsize=(8, 5)
)

plt.xlabel("Fare")
plt.title("Fare Distribution")
plt.show()
```

Ask whether the distribution appears symmetrical or skewed.

Part IV - Feature Engineering

71. What is Feature Engineering?

Feature engineering means creating useful variables from existing information.

Raw values:

```
sibsp = 1
parch = 2
```

can be transformed into:

```
family_size = 4
```

because the family group includes the passenger themselves plus siblings/spouses and parents/children.

Feature engineering can improve both analysis and machine-learning representation.

72. Feature Engineering 1 - Family Size

```
data["family_size"] = (  
    data["sibsp"] +  
    data["parch"] +  
    1  
)
```

Inspect:

```
data[[  
    "sibsp",  
    "parch",  
    "family_size"  
]].head()
```

73. Family Size and Survival

```
data.groupby("family_size")["survived"].agg([  
    "count",  
    "mean"  
)
```

This helps investigate whether travelling in different-sized family groups is associated with survival.

74. Feature Engineering 2 - Travelling Alone

```
data["is_alone_engineered"] = (  
    data["family_size"] == 1  
) .astype(int)
```

Check:

```
data[[  
    "family_size",  
    "is_alone_engineered"  
]].head()
```

75. Compare with Existing alone

The source dataset already contains an alone feature.

```
data["alone"]
```

Compare it with our engineered feature:

```
pd.crosstab(  
    data["alone"],  
    data["is_alone_engineered"]  
)
```

This demonstrates that a dataset may contain features that can be reconstructed from other variables.

76. Feature Engineering 3 - Age Groups

A raw age like 27 may be useful, but a category such as Young Adult may also be useful.

```
bins = [0, 12, 18, 35, 50, 65, np.inf]
```

```
labels = [
    "Child",
    "Teen",
    "Young Adult",
    "Adult",
    "Older Adult",
    "Senior"
]

data["age_group"] = pd.cut(
    data["age"],
    bins=bins,
    labels=labels
)
```

77. Age Group Distribution

```
data["age_group"].value_counts()
```

Survival rate:

```
data.groupby(
    "age_group",
    observed=False
)["survived"].mean()
```

78. Feature Engineering 4 - Child

Create a simple binary feature:

```
data["is_child"] = (
    data["age"] < 16
).astype(int)
```

Investigate:

```
data.groupby("is_child")["survived"].mean()
```

79. Feature Engineering 5 - Family Type

Instead of exact family size, create a broader category:

```
def family_type(size):
    if size == 1:
        return "Alone"
    elif size <= 4:
        return "Small Family"
    else:
        return "Large Family"
```

Apply it:

```
data["family_type"] = (  
    data["family_size"]  
    .apply(family_type)  
)
```

Inspect:

```
data[[  
    "family_size",  
    "family_type"  
]].head(10)
```

80. Family Type and Survival

```
data.groupby(  
    "family_type"  
)[ "survived" ].agg([  
    "count",  
    "mean"  
)
```

81. Feature Engineering 6 - Fare Per Person

A simple derived feature is:

```
data["fare_per_person"] = (  
    data["fare"] /  
    data["family_size"]  
)
```

Inspect:

```
data[[  
    "fare",  
    "family_size",  
    "fare_per_person"  
]].head()
```

This is only a proxy. The dataset does not directly tell us exactly how each ticket fare was shared.

82. Feature Engineering 7 - Fare Bands

Instead of exact fare values:

```
data["fare_band"] = pd.qcut(  
    data["fare"],  
    q=4,  
    duplicates="drop"  
)
```

Check:

```
data["fare_band"].value_counts()
```

Survival by band:

```
data.groupby(  
    "fare_band",
```

```
observed=False  
)["survived"].mean()
```

83. cut() vs qcut()

cut()

Creates bins based on chosen value ranges, for example:

```
0-10  
10-20  
20-30
```

qcut()

Creates quantile-based groups with approximately similar numbers of observations, for example:

```
bottom 25%  
25%-50%  
50%-75%  
top 25%
```

84. Feature Engineering 8 - Fare Log Transformation

Fare is highly skewed. A common transformation is:

```
data["log_fare"] = np.log1p(  
    data["fare"]  
)
```

`np.log1p(x)` calculates $\log(1 + x)$ and works safely when $x = 0$.

85. Compare Fare Distribution

Original:

```
data["fare"].hist(bins=40)  
plt.title("Original Fare")  
plt.show()
```

Transformed:

```
data["log_fare"].hist(bins=40)  
plt.title("Log Transformed Fare")  
plt.show()
```

Students can visually compare the shape and skewness.

86. Categorical Data

Machine-learning algorithms usually expect numeric inputs. But the dataset contains values such as:

```
male  
female
```

and:

S
C
Q

We therefore need numeric representations.

87. Binary Encoding

For sex:

```
data["sex_encoded"] = (  
    data["sex"]  
    .map({  
        "male": 0,  
        "female": 1  
    })  
)
```

Inspect:

```
data[[  
    "sex",  
    "sex_encoded"  
]].head()
```

88. One-Hot Encoding

For embarkation:

```
pd.get_dummies(  
    data["embarked"],  
    dtype=int  
)
```

Or directly modify a model dataset:

```
model_data = pd.get_dummies(  
    data,  
    columns=["embarked"],  
    dtype=int  
)
```

89. Understanding Redundant Features

The Titanic dataset is excellent for teaching redundancy.

Consider:

```
pclass  
class
```

Both represent passenger class.

Similarly:

```
embarked  
embark_town
```

represent nearly the same information in different forms.

And alone can be derived from sibsp + parch.

Redundant features are not automatically wrong, but we should understand where information comes from.

90. The Most Important Example - Data Leakage

Inspect:

```
data[[  
  "survived",  
  "alive"  
]].head(20)
```

survived uses 0/1, while alive uses no/yes. They represent the same outcome.

If the machine-learning target is survived and we provide alive as an input, the model effectively already knows the answer. This is **target leakage**.

91. Leakage vs Redundancy

Target leakage

alive -> effectively gives away survived

This is a serious modelling problem.

Redundancy

class <-> pclass

Both describe passenger class.

Derived information

alone <- sibsp + parch

and conceptually:

adult_male <- age + sex logic

Derived features may contain information already available elsewhere.

92. Build a Clean Working Dataset

For teaching and model preparation, start from relatively fundamental fields:

```
clean = df[  
  [  
    "survived",  
    "pclass",  
    "sex",  
    "age",  
    "sibsp",  
    "parch",  
    "fare",  
    "embarked"  
  ]  
].copy()
```

93. Add Missing-Value Indicator

```
clean["age_missing"] = (  
    clean["age"]  
    .isna()  
    .astype(int)  
)
```

94. Impute Age

```
age_medians = clean.groupby(  
    ["pclass", "sex"]  
)[ "age"].transform("median")  
  
clean["age"] = clean["age"].fillna(  
    age_medians  
)
```

95. Fill Embarkation

```
clean["embarked"] = clean["embarked"].fillna(  
    clean["embarked"].mode()[0]  
)
```

96. Add Family Size

```
clean["family_size"] = (  
    clean["sibsp"] +  
    clean["parch"] +  
    1  
)
```

97. Add Alone Feature

```
clean["is_alone"] = (  
    clean["family_size"] == 1  
) .astype(int)
```

98. Add Child Feature

```
clean["is_child"] = (  
    clean["age"] < 16  
) .astype(int)
```

99. Add Fare Per Person

```
clean["fare_per_person"] = (  
    clean["fare"] /  
    clean["family_size"]  
)
```

100. Add Age Groups

```
clean["age_group"] = pd.cut(
    clean["age"],
    bins=[0, 12, 18, 35, 50, 65, np.inf],
    labels=[
        "Child",
        "Teen",
        "Young Adult",
        "Adult",
        "Older Adult",
        "Senior"
    ]
)
```

101. Inspect the Engineered Dataset

```
clean.head()
clean.info()
```

The table is now richer than the original subset. Feature engineering has transformed existing information into additional representations that may be useful for analysis or modelling.

Part V - Hypothesis-Driven Data Analysis

102. What is a Hypothesis?

Suppose we notice:

```
clean.groupby("sex")["survived"].mean()
```

The group values may look different. But we should ask:

Is this difference potentially just random variation under a simple null model?

This introduces statistical hypothesis testing.

103. Scientific Thinking

Instead of simply saying:

Females survived more.

ask:

Is survival associated with sex in this dataset?

The analytical flow becomes:

Observation

↓

Question

↓

Hypothesis

↓

Statistical Test

↓

Evidence

↓

Interpretation

104. Null Hypothesis

Usually written as H_0 or H_o .

The null hypothesis generally represents a baseline such as no association or no difference.

Example:

H_0 :

Passenger sex and survival are independent.

105. Alternative Hypothesis

Usually written as H_1 or H_a .

Example:

H_1 :

Passenger sex and survival are associated.

106. What is a p-value?

At a beginner level, think of the p-value as measuring how compatible the observed result is with the null hypothesis and the assumptions of the statistical test.

A common teaching threshold is:

$\alpha = 0.05$

If:

$p < 0.05$

we reject the null hypothesis at that chosen threshold.

If:

$p \geq 0.05$

we do **not** reject the null hypothesis.

Important: failing to reject H_0 does **not** prove that H_0 is true.

107. Statistical Significance is Not Practical Importance

A small p-value does not automatically mean that an effect is large or practically important.

Also consider:

- effect size
- actual percentages or group differences
- domain context
- sample size
- test assumptions
- confounding variables

108. Hypothesis 1 - Sex and Survival

Question:

Is passenger sex associated with survival?

Both variables are categorical:

sex
survived

A common test is the **Chi-Square Test of Independence**.

109. Create the Contingency Table

```
sex_survival_table = pd.crosstab(  
    clean["sex"],  
    clean["survived"]  
)
```

sex_survival_table

110. Run the Chi-Square Test

```
from scipy.stats import chi2_contingency  
  
chi2, p_value, dof, expected = chi2_contingency(  
    sex_survival_table  
)  
  
print("Chi-square:", chi2)  
print("p-value:", p_value)  
print("Degrees of freedom:", dof)
```

111. Interpret the Test

```
alpha = 0.05  
  
if p_value < alpha:  
    print("Reject H0")  
    print("Evidence of an association between sex and survival.")  
else:  
    print("Do not reject H0")  
    print("Insufficient evidence of an association.")
```

Do **not** interpret this as proving that sex caused survival. This is observational historical data. The test investigates association.

112. Effect Size - Cramer's V

Statistical significance alone is not enough. For a contingency table, one effect-size measure is Cramer's V.

```
n = sex_survival_table.to_numpy().sum()  
rows, cols = sex_survival_table.shape  
  
cramers_v = np.sqrt(  
    (chi2 / n) * (1 / min(rows, cols))
```

```

chi2 /
(
    n *
    min(rows - 1, cols - 1)
)
)

```

```
print("Cramer's V:", cramers_v)
```

This provides additional information about the strength of association.

113. Hypothesis 2 - Passenger Class and Survival

Question:

Is passenger class associated with survival?

Hypotheses:

H0:

Passenger class and survival are independent.

H1:

Passenger class and survival are associated.

Create the table:

```

class_survival = pd.crosstab(
    clean["pclass"],
    clean["survived"]
)

```

Run the test:

```

chi2, p_value, dof, expected = chi2_contingency(
    class_survival
)

```

```

print("Chi-square:", chi2)
print("p-value:", p_value)

```

114. Do Not Stop at the p-value

Inspect the actual survival rates:

```

clean.groupby(
    "pclass"
)["survived"].mean()

```

And percentages:

```

pd.crosstab(
    clean["pclass"],
    clean["survived"],
    normalize="index"
) * 100

```

Statistical tests should complement descriptive understanding, not replace it.

115. Hypothesis 3 - Age and Survival

Question:

Was passenger age different between survivors and non-survivors?

age is numerical, while survived creates two groups.

Create the groups:

```
survived_age = clean.loc[
    clean["survived"] == 1,
    "age"
]

not_survived_age = clean.loc[
    clean["survived"] == 0,
    "age"
]
```

116. Compare Descriptive Statistics First

```
print(
    "Survivor mean:",
    survived_age.mean()
)

print(
    "Non-survivor mean:",
    not_survived_age.mean()
)

print(
    "Survivor median:",
    survived_age.median()
)

print(
    "Non-survivor median:",
    not_survived_age.median()
)
```

Do not jump immediately to a statistical test. First understand the group distributions.

117. Welch's t-test

Hypotheses:

H0:

Mean age is equal between survivors and non-survivors.

H1:

Mean age differs between survivors and non-survivors.

Run:

```
from scipy.stats import ttest_ind

t_stat, p_value = ttest_ind(
```

```

    survived_age,
    not_survived_age,
    equal_var=False
)

print("t-statistic:", t_stat)
print("p-value:", p_value)

```

Using `equal_var=False` performs Welch's t-test, which does not assume equal population variances.

118. Important Assumptions

Before blindly using a t-test, discuss:

- independence of observations
- distribution shape
- extreme outliers
- sample size
- whether a comparison of means is meaningful

Statistical tests should be chosen because they fit the analytical question and assumptions, not simply because we know their names.

119. Hypothesis 4 - Fare and Survival

First compare descriptive statistics:

```

clean.groupby(
    "survived"
)["fare"].agg([
    "mean",
    "median",
    "std"
])

```

Then visualize:

```

clean[
    clean["survived"] == 1
]["fare"].hist(
    bins=40,
    alpha=0.5,
    label="Survived"
)

```

```

clean[
    clean["survived"] == 0
]["fare"].hist(
    bins=40,
    alpha=0.5,
    label="Did not survive"
)

```

```

plt.legend()
plt.xlabel("Fare")
plt.title("Fare Distribution by Survival")
plt.show()

```

Fare is strongly skewed, which is useful for discussing why test choice matters.

120. Mann-Whitney U Test

For a heavily skewed variable, we can demonstrate a non-parametric rank-based comparison.

```
from scipy.stats import mannwhitneyu
```

```
survived_fare = clean.loc[
    clean["survived"] == 1,
    "fare"
]

not_survived_fare = clean.loc[
    clean["survived"] == 0,
    "fare"
]

u_stat, p_value = mannwhitneyu(
    survived_fare,
    not_survived_fare,
    alternative="two-sided"
)
```

```
print("U statistic:", u_stat)
print("p-value:", p_value)
```

Do not interpret this as proving that survivors paid more because of survival. The test evaluates differences in the distributions/ranks under its assumptions.

121. Hypothesis 5 - Travelling Alone and Survival

We created `is_alone`. Now test it.

```
alone_survival = pd.crosstab(
    clean["is_alone"],
    clean["survived"]
)

alone_survival
Chi-square:
chi2, p_value, dof, expected = chi2_contingency(
    alone_survival
)

print("p-value:", p_value)
```

Then inspect actual rates:

```
clean.groupby(
    "is_alone"
)["survived"].mean()
```

122. Feature Engineering Can Create New Hypotheses

Raw variables:

```
sibsp
parch
```

created:

family_size

Then family_size can be transformed into:

family_type

Now we can ask:

Is family type associated with survival?

Feature engineering is therefore more than preparing inputs for machine learning. It can help us ask better questions about the data.

123. Test Family Type and Survival

Create categories:

```
clean["family_type"] = pd.cut(
    clean["family_size"],
    bins=[0, 1, 4, np.inf],
    labels=[
        "Alone",
        "Small Family",
        "Large Family"
    ]
)
```

Table:

```
family_survival = pd.crosstab(
    clean["family_type"],
    clean["survived"]
)
```

family_survival

Test:

```
chi2, p_value, dof, expected = chi2_contingency(
    family_survival
)
```

```
print("p-value:", p_value)
```

124. Association Does Not Mean Causation

Suppose we observe:

higher fare <-> higher survival

We must not immediately conclude:

paying more money caused survival

Fare is related to several other characteristics, including passenger class, ticket arrangements, cabin location, wealth, and other passenger differences.

These connected variables may act as **confounding factors**.

125. Example of Confounding

Suppose Fare -> Survival appears strong. But:

Fare <-> Passenger Class

Passenger Class <-> Survival

Passenger class may explain part of the observed relationship.

Check:

```
clean.groupby(  
    "pclass"  
)["fare"].median()
```

Then:

```
clean.groupby(  
    "pclass"  
)["survived"].mean()
```

Real data analysis becomes more useful when we look beyond a single pair of variables.

126. Ask Better Questions

Beginner question:

Did fare affect survival?

Better analytical question:

Is fare associated with survival in this dataset?

Even better:

Does the relationship between fare and survival remain after considering passenger class and other variables?

That final question leads naturally toward regression, statistical modelling, and machine learning.

127. Correlation

Select numerical variables:

```
numeric_columns = [  
    "survived",  
    "pclass",  
    "age",  
    "sibsp",  
    "parch",  
    "fare",  
    "family_size",  
    "fare_per_person"  
]
```

```
correlations = clean[  
    numeric_columns  
].corr()
```

```
correlations
```

128. Correlation with Survival

```
correlations[  
    "survived"  
].sort_values(  
    ascending=False  
)
```

Important reminders:

- Correlation does not prove causation.
- Pearson correlation primarily describes linear association between numerical representations.
- Different measures may be better for categorical variables or nonlinear relationships.

Part VI - From Analysis to Machine-Learning Data

129. A Complete Data Analysis Workflow

At this point students should understand this process:

1. Define the question
2. Get the data
3. Inspect the data
4. Understand column meanings
5. Check data types
6. Check missing values
7. Clean data
8. Explore distributions
9. Compare groups
10. Engineer useful features
11. Form hypotheses
12. Select appropriate tests
13. Calculate statistics
14. Interpret results
15. Consider confounders and limitations
16. Prepare features for modelling

130. Prepare a Machine-Learning Dataset

```
ml_data = clean[  
    [  
        "survived",  
        "pclass",  
        "sex",  
        "age",  
        "fare",  
        "embarked",  
        "family_size",  
        "is_alone",  
        "is_child",  
        "fare_per_person",  
        "age_missing"  
    ]  
].copy()
```

131. Encode Categorical Variables

```
ml_data = pd.get_dummies(  
    ml_data,  
    columns=[  
        "sex",  
        "embarked"  
    ],  
    drop_first=True,  
    dtype=int  
)
```

Inspect:

```
ml_data.head()
```

132. Check Everything is Numerical

```
ml_data.dtypes
```

At this stage, the dataset is much closer to something that could be passed to a machine-learning algorithm.

133. Separate Features and Target

Target:

```
y = ml_data["survived"]
```

Features:

```
X = ml_data.drop(  
    columns="survived"  
)
```

Check:

```
X.head()  
y.head()
```

This introduces the basic machine-learning structure:

```
X = input features  
y = target/output
```

134. What Did We Actually Do?

We started with a raw CSV and moved through:

```
Data Loading  
    ↓  
Inspection  
    ↓  
NumPy Operations  
    ↓  
Pandas Operations  
    ↓  
Filtering  
    ↓  
Grouping
```

↓
Missing Value Analysis
↓
Cleaning
↓
EDA
↓
Feature Engineering
↓
Hypothesis Generation
↓
Statistical Testing
↓
Model-Ready Features

That is a major part of a real data-science workflow.

135. Important NumPy Commands Learned

```
np.array()  
np.mean()  
np.median()  
np.std()  
np.min()  
np.max()  
np.sum()  
np.nan  
np.nanmean()  
np.nanmedian()  
np.where()  
np.log1p()
```

Plus the concepts of:

- indexing
- slicing
- Boolean masks
- vectorization

136. Important Pandas Commands Learned

```
pd.read_csv()  
  
df.head()  
df.tail()  
df.sample()  
  
df.shape  
df.columns  
df.dtypes  
  
df.info()  
df.describe()  
  
df["column"]  
df[["column1", "column2"]]
```

```

df.loc[]
df.iloc[]

df.isna()
df.fillna()
df.dropna()

df.value_counts()
df.groupby()
df.agg()
df.sort_values()

pd.crosstab()
pd.cut()
pd.qcut()
pd.get_dummies()

```

137. Important Statistics Learned

Students should now recognize:

- mean
- median
- standard deviation
- distributions
- percentages and rates
- conditional comparisons
- correlation
- null hypothesis
- alternative hypothesis
- p-value
- significance level
- chi-square test
- Welch's t-test
- Mann-Whitney U test
- effect size
- confounding
- association vs causation

138. The Most Important Thinking Skill

The goal is **not** to memorize:

```
df.groupby(...)
```

The important skill is converting a question into data operations.

Example:

Question:

Did women have different survival rates?

↓

Variables:

sex + survived

↓

Data operation:

groupby / crosstab

↓

Statistical question:

Are sex and survival associated?

↓

Statistical test:

Chi-square

↓

Interpretation:

Look at p-value + proportions + effect size + context

That is **data thinking**.

Part VII - Guided Student Exercises

139. Guided Student Exercise

Students should answer the following without being given the complete solution code.

Exercise 1

How many passengers were male and how many were female?

Exercise 2

Calculate the overall survival percentage.

Exercise 3

Calculate survival percentage by sex.

Exercise 4

Calculate survival percentage by passenger class.

Exercise 5

Find the average age by survival group.

Exercise 6

Find passengers who:

were female

AND

were first class

AND

survived

Exercise 7

Create family_size using sibsp and parch.

Exercise 8

Create `is_child` where age is less than 16.

Exercise 9

Create three age categories of your own, such as:

Young
Adult
Senior

Exercise 10

Create a hypothesis about embarked and survival.

Write:

H_0
 H_1

Then choose an appropriate statistical test.

140. Advanced Student Challenge

Investigate the claim:

First-class passengers survived more frequently than third-class passengers.

Students must provide:

1. Relevant descriptive statistics
2. Survival percentages
3. A visualization
4. Null hypothesis
5. Alternative hypothesis
6. Statistical test
7. p-value
8. Interpretation
9. At least one possible confounding factor
10. A statement explaining why association does not prove causation

141. Feature Engineering Challenge

Students must create at least **five new features**.

Possible examples:

`family_size`
`is_alone`
`is_child`
`age_group`
`fare_per_person`
`fare_band`
`log_fare`
`age_missing`
`deck_known`
`family_type`

For every feature, explain:

1. Original columns used
2. Formula or transformation
3. Why the feature might be useful
4. What information could be lost
5. Whether the feature duplicates existing information

142. Hypothesis Challenge

Students must create three hypotheses.

Hypothesis A

Question: Is passenger class associated with survival?

H0: Passenger class and survival are independent.

H1: Passenger class and survival are associated.

Test: Chi-square test.

Hypothesis B

Question: Does passenger age differ between survivors and non-survivors?

H0: The groups do not differ in the population quantity targeted by the selected test.

H1: A difference exists.

Possible test: Welch's t-test or another appropriate comparison after checking assumptions.

Hypothesis C

Students must create their own using one or more of:

family_size
is_alone
embarked
fare
age_group
is_child

143. Final Mini Project - Titanic Data Investigation

Each student should submit one Jupyter Notebook or Google Colab notebook containing the following sections.

Section 1 - Dataset

- Download Titanic CSV
- Load it with pandas
- Explain dataset shape
- Explain important columns

Section 2 - NumPy

Demonstrate:

- arrays
- indexing
- slicing
- vectorization
- Boolean filtering
- mean
- median
- standard deviation
- missing-value handling

Section 3 - Pandas

Demonstrate:

- Series
- DataFrames
- filtering
- sorting
- loc
- iloc
- value_counts
- groupby
- aggregation
- crosstab

Section 4 - Data Cleaning

Investigate:

- missing values
- missing percentages
- age
- embarked
- deck
- duplicates

Explain every cleaning decision.

Section 5 - EDA

Answer at least five questions. For example:

What percentage survived?

How did survival differ by sex?

How did survival differ by class?

What was the age distribution?

How was fare distributed?

Section 6 - Feature Engineering

Create at least five new features.

Section 7 - Hypothesis Testing

Test at least three hypotheses.

For each hypothesis provide:

Research Question

H0

H1

Statistical Test

Test Statistic

p-value

Decision at chosen alpha

Interpretation

Limitation

Section 8 - Conclusions

Explain:

What patterns were observed?

Which features appear useful?

What statistical relationships were found?

What can we NOT conclude?

What variables would you use in a future ML model?

144. Final Lesson

Data science is not simply:

CSV

↓

Machine Learning Model

↓

Prediction

A better workflow is:

Problem

↓

Question

↓

Data

↓

Understanding

↓

Cleaning

↓

Exploration

↓

Feature Engineering

↓

Hypothesis

↓

Evidence

↓

Model

↓

Evaluation

↓

Decision

The model is only one part of the process.

145. Main Takeaway

A beginner often asks:

Which machine-learning algorithm should I use?

A better first set of questions is:

What does my data mean?

What is missing?

What relationships exist?

Which variables represent the same information?

Could any variable leak the answer?

What useful features can I create?

What hypotheses can I investigate?

What evidence supports those hypotheses?

What assumptions am I making?

Once students begin asking these questions, they are moving from simply writing Python code toward **thinking like data and AI practitioners**.

Instructor Emphasis

Spend extra teaching time on **Sections 71-126**. This is the transition from “I know pandas commands” to “I can investigate data, engineer useful information, identify leakage, formulate a hypothesis, choose a test, and interpret evidence responsibly.”

AI Foundation Program - Titanic Data Lab

NumPy, Pandas, Feature Engineering, and Hypothesis Testing